↑ Jump to Table of Contents← Collapse Sidebar

Micropub

W3C Recommendation 23 May 2017

This version:
https://www.w3.org/TR/2017/REC-micropub-20170523/
Latest published version:
https://www.w3.org/TR/micropub/
Latest editor's draft:
https://micropub.net/draft/
Test suite:
https://micropub.rocks/
Implementation report:
https://micropub.net/implementation-reports/
Previous version:
https://www.w3.org/TR/2017/PR-micropub-20170413/
Editor:
Aaron Parecki
Repository:
Github
Issues
Commits

Please check the **errata** for any errors or issues reported since publication.

The English version of this specification is the only normative version. Non-normative translations may also be available.

Copyright © 2017 W3C® (MIT, ERCIM, Keio, Beihang). W3C liability, trademark and permissive document license rules apply.

Abstract

The Micropub protocol is used to create, update and delete posts on one's own domain using third-party clients. Web apps and native apps (e.g., iPhone, Android) can use Micropub to post and edit articles, short notes, comments, likes, photos, events or other kinds of posts on your own website.

Author's Note

This section is non-normative.

This specification was contributed to the W3C from the IndieWeb community. More history and evolution of Micropub can be found on the IndieWeb wiki.

Status of This Document

This section describes the status of this document at the time of its publication. Other documents may supersede this document. A list of current W3C publications and the latest revision of this technical report can be found in the W3C technical reports index at https://www.w3.org/TR/.

This document was published by the Social Web Working Group as a Recommendation. Comments regarding this document are welcome. Please send them to public-socialweb@w3.org (subscribe, archives).

Please see the Working Group's implementation report.

This document has been reviewed by W3C Members, by software developers, and by other W3C groups and interested parties, and is endorsed by the Director as a W3C Recommendation. It is a stable document and may be used as reference material or cited from another document. W3C's role in making the Recommendation is to draw attention to the specification and to promote its widespread deployment. This enhances the functionality and interoperability of the Web.

This document was produced by a group operating under the 5 February 2004 W3C Patent Policy. W3C maintains a public list of any patent disclosures made in connection with the deliverables of the group; that page also includes instructions for disclosing a patent. An individual who has actual knowledge of a patent which the individual believes contains Essential Claim(s) must disclose the information in accordance with section 6 of the W3C Patent Policy.

This document is governed by the 1 March 2017 W3C Process Document.

Table of Contents

1. 1. Introduction
    1. 1.1 Social Web Working Group
    2. 1.2 Background
2. 2. Conformance
    1. 2.1 Conformance Classes
        1. 2.1.1 Publishing Clients
        2. 2.1.2 Editing Clients
        3. 2.1.3 Servers
    2. 2.2 Candidate Recommendation Exit Criteria
        1. 2.2.1 Client
        2. 2.2.2 Server
        3. 2.2.3 Independent
        4. 2.2.4 Interoperable
        5. 2.2.5 Implementation
        6. 2.2.6 Feature
    3. 2.3 Test Suite and Reporting
3. 3. Syntax
    1. 3.1 Overview
        1. 3.1.1 Form-Encoded and Multipart Requests
    2. 3.2 Reserved Properties
    3. 3.3 Create
        1. 3.3.1 Uploading Files
        2. 3.3.2 JSON Syntax
        3. 3.3.3 Nested Microformats Objects
        4. 3.3.4 Bidirectional Text
        5. 3.3.5 Unrecognized Properties

6. 3.3.6 Response
4. 3.4 Update
1. 3.4.1 Replace
2. 3.4.2 Add
3. 3.4.3 Remove
4. 3.4.4 Response
5. 3.5 Delete
1. 3.5.1 Response
6. 3.6 Media Endpoint
1. 3.6.1 Discovery
2. 3.6.2 Authentication
3. 3.6.3 Request
4. 3.6.4 Response
1. 3.6.4.1 Media Endpoint Error Response
7. 3.7 Querying
1. 3.7.1 Configuration
2. 3.7.2 Source Content
3. 3.7.3 Syndication Targets
4. 3.7.4 Query Extensions
8. 3.8 Error Response
4. 4. Vocabulary
1. 4.1 Examples of Creating Objects
1. 4.1.1 New Note
2. 4.1.2 New Reply
3. 4.1.3 New Article with HTML
4. 4.1.4 New Article with Embedded Images
5. 4.1.5 Posting Files
5. 5. Authentication and Authorization
1. 5.1 Authentication
2. 5.2 Authorization
3. 5.3 Endpoint Discovery
4. 5.4 Scope
6. 6. Security Considerations
1. 6.1 External Content
2. 6.2 Security and Privacy Review
7. 7. IANA Considerations
8. A. Resources
1. A.1 Implementations
9. B. Acknowledgements
10. C. Change Log
1. C.1 Changes from 13 April 2017 PR to this version
2. C.2 Changes from 18 October 2016 CR to 13 April 2017 PR
3. C.3 Changes from 16 August 2016 CR to 18 October 2016 CR
4. C.4 Changes from 13 July 2016 WD to 16 August 2016 CR
5. C.5 Changes from 21 June 2016 WD to 13 July 2016 WD
6. C.6 Changes from 04 May 2016 WD to 21 June 2016 WD
7. C.7 Changes from 01 March 2016 WD to 04 May 2016 WD
8. C.8 Changes from 28 January 2016 FPWD to 01 March 2016 WD
11. D. References
1. D.1 Normative references
2. D.2 Informative references

1. Introduction

This section is non-normative.

Micropub is a spec to create, update and delete posts on a server using web or native app clients. Micropub is primarily focused around creating "posts" (individual pieces of content such as blog posts, photos, short notes, comments, and more) on a website, although it can be used for other kinds of content as well. The Micropub spec defines a simple mechanism to create content, as well as a more thorough mechanism to update and delete content.

1.1 Social Web Working Group

Micropub is one of several related specifications being produced by the Social Web Working Group. Implementers interested in alternative approaches and complementary protocols should review ActivityPub as well as the overview document [social-web-protocols].

1.2 Background

The Micropub specification began as a simplified version of the AtomPub and MetaWeblog APIs. Where AtomPub is an API to create items in an Atom feed, Micropub is an API to create items in a [Microformats2] feed.

In addition to being used with a different vocabulary from AtomPub and MetaWeblog, Micropub simplifies and improves upon both APIs in a number of ways. Micropub uses OAuth 2.0 Bearer Tokens for authentication, rather than the previous insecure username/password authentication method. Micropub also uses traditional form posts as well as JSON posts, which is both simpler and more secure than an XMLRPC approach.

The Micropub vocabulary is derived directly from the [Microformats2] vocabulary. Micropub is meant to be a serialization of Microformats that can be submitted as an HTTP POST. The method for developing new Micropub vocabularies is to look at the Microformats representation and work backwards.

2. Conformance

The key words "must", "must not", "required", "shall", "shall not", " should", "should not", "recommended", "may", and "optional" in this document are to be interpreted as described in [RFC2119].

2.1 Conformance Classes

This section describes the conformance criteria for Micropub clients and servers. All implementations must support UTF-8 encoding.

2.1.1 Publishing Clients

A conforming Micropub client that creates posts:

- must support sending `x-www-form-urlencoded` requests
- must support the [h-entry] vocabulary
- If the client creates posts by uploading file attachments, it must check for the presence of a Media Endpoint and if present, send the file there instead of to the Micropub endpoint
- should handle server error messages gracefully, presenting helpful messages to the user
- should support endpoint discovery

2.1.2 Editing Clients

A conforming Micropub client that edits posts:

- must support sending JSON-encoded requests
- must support the [h-entry] vocabulary

2.1.3 Servers

A conforming Micropub server:

- must support both header and form parameter methods of authentication
- must support creating posts with the [h-entry] vocabulary
- must support creating posts using the `x-www-form-urlencoded` syntax
- should support updating and deleting posts
- Servers that support updating posts must support JSON syntax and the source content query
- Servers that do not specify a Media Endpoint must support `multipart/form-data` requests for creating posts
- Servers that specify a Media Endpoint must support the configuration query, other servers should support the configuration query

2.2 Candidate Recommendation Exit Criteria

This specification exited CR by there being least two independent, interoperable implementations of each feature. Each feature may be implemented by a different set of products. There is no requirement that all features be implemented by a single product. For the purposes of this criterion, we define the following terms:

2.2.1 Client

A Micropub Client is an implementation that sends Micropub requests to create or otherwise manipulate posts. The conformance criteria are described in Conformance Classes above.

2.2.2 Server

A Micropub Server is an implementation that can create and optionally edit and delete posts given a Micropub request. The Micropub server may also support a Media Endpoint for handling file uploads outside of the primary Micropub endpoint. The conformance criteria are described in Conformance Classes above.

2.2.3 Independent

Each implementation must be developed by a different party and cannot share, reuse, or derive from code used by another qualifying implementation. Sections of code that have no bearing on the implementation of this specification are exempt from this requirement.

2.2.4 Interoperable

A Client and Server implementation are considered interoperable for a specific feature when the Server takes the defined action that the Client requests, the Client gets the expected response from a Server according to the feature, and the Server sends the expected response to the Client.

2.2.5 Implementation

An Implementation is a Micropub Client or Server which meets all of the following criteria:

- implements the corresponding conformance class of the specification
- is available to the general public, as downloadable software or as a hosted service
- is not experimental (i.e. is intended for a wide audience and could be used on a daily basis)
- is suitable for a person to use as his/her primary implementation on a website

2.2.6 Feature

For the purposes of evaluating exit criteria, each of the following is considered a feature:

- Discovering the Micropub endpoint given the profile URL of a user
- Authenticating requests by including the access token in the HTTP `Authorization` header
- Authenticating requests by including the access token in the post body for `x-www-form-urlencoded` requests
- Limiting the ability to create posts given an access token by requiring that the access token contain at least one OAuth 2.0 scope value
- Creating a post using `x-www-form-urlencoded` syntax with one or more properties
- Creating a post using JSON syntax with one or more properties
- Creating a post using `x-www-form-urlencoded` syntax with multiple values of the same property name
- Creating a post using JSON syntax with multiple values of the same property name
- Creating a post using JSON syntax including a nested Microformats2 object
- Uploading a file to the specified Media Endpoint
- Creating a post with a file by sending the request as `multipart/form-data` to the Micropub endpoint
- Creating a post with a photo referenced by URL
- Creating a post with a photo referenced by URL that includes image alt text
- Creating a post where the request contains properties the server does not recognize
- Returning `HTTP 201 Created` and a `Location` header when creating a post
- Returning `HTTP 202 Created` and a `Location` header when creating a post
- Updating a post and replacing all values of a property
- Updating a post and adding a value to a property
- Updating a post and removing a value from a property
- Updating a post and removing a property
- Returning `HTTP 200 OK` when updating a post
- Returning `HTTP 201 Created` when updating a post if the update cause the post URL to change
- Returning `HTTP 204 No Content` when updating a post
- Deleting a post using `x-www-form-urlencoded` syntax
- Deleting a post using JSON syntax
- Undeleting a post using `x-www-form-urlencoded` syntax
- Undeleting a post using JSON syntax
- Uploading a photo to the Media Endpoint and using the resulting URL when creating a post
- Querying the Micropub endpoint with `q=config` to retrieve the Media Endpoint and syndication targets if specified
- Querying the Micropub endpoint with `q=syndicate-to` to retrieve the list of syndication targets
- Querying the Micropub endpoint for a post's source content without specifying a list of properties
- Querying the Micropub endpoint for a post's source content looking only for specific properties

2.3 Test Suite and Reporting

Please submit your implementation reports at https://micropub.net/implementation-reports/. Instructions are provided at the URL. The implementation report template references the tests available at micropub.rocks.

micropub.rocks provides many test cases you can use to live-test your implementation. It also is a good tool to use while developing a Micropub implementation, as it provides detailed responses when errors are encountered.

3. Syntax

As [microformats2-parsing] has a relatively small ruleset for parsing HTML documents into a data structure, Micropub similarly defines a small set of rules to interpret HTTP POST and GET requests as Micropub commands. Where [microformats2-parsing] does not require changing the parsing rules to introduce new properties of an object such as an [h-entry], Micropub similarly does not require changing parsing rules to interpret requests that may correspond to different post types, such as posting videos vs "likes".

The Micropub syntax describes how to interpret HTTP POST and GET requests into useful server actions.

3.1 Overview

All Micropub requests to create posts are sent as UTF-8 `x-www-form-urlencoded`, `multipart/form-data` [HTML5], or [JSON]-encoded HTTP requests. Responses typically do not include a response body, indicating the needed information (such as the URL of the created post) in HTTP headers. When a response body is necessary, it should be returned as a [JSON] encoded object.

3.1.1 Form-Encoded and Multipart Requests

For `x-www-form-urlencoded` and `multipart/form-data` requests, Micropub supports an extension of the standard URL encoding that includes explicit indicators of multi-valued properties. Specifically, this means in order to send multiple values for a given property, you must append square brackets `[]` to the property name.

For example, to specify multiple values for the property "category", the request would include `category[]=foo&category[]=bar`.

On the server side, it is expected that the server will convert this to an internal representation of an array. For example, the equivalent JSON representation of this would be:

```
{
  "category": [
    "foo",
    "bar"
  ]
}
```

This works equally well in multipart requests, where each value is given in a separate "part" of the request, and the name is given in a line such as `Content-Disposition: form-data; name="category[]"`.

Note that the extent of the extensions to the `x-www-form-urlencoded` syntax is the addition of the square brackets to indicate an array. Syntax such as `foo[0]` and `foo[bar]` is not supported, and so clients are expected to use the JSON syntax when posting more complex objects.

3.2 Reserved Properties

A few POST body property names are reserved when requests are sent as `x-www-form-urlencoded` or `multipart/form-data`.

- `access_token` - the OAuth Bearer token authenticating the request (the access token may be sent in an HTTP Authorization header or this form parameter)
- `h` - used to specify the object type being created
- `action` - indicates whether this is an `delete`, or `undelete` (updates are not supported in the form-encoded syntax)

- `url` - indicates the URL of the object being acted on
- `mp-*` - reserved for future use

When creating posts using `x-www-form-urlencoded` or `multipart/form-data` requests, all other properties in the request are considered properties of the object being created.

The server must not store the `access_token` property in the post.

Properties beginning with `mp-` are reserved as a mechanism for clients to give commands to servers. Where typically properties of a post are visible to users, commands to the server are not visible to users, so it does not make sense to set a property on the post for a command. Clients and servers wishing to experiment with creating new `mp-` commands are encouraged to brainstorm and document implementations at indieweb.org/Micropub-extensions.

When creating posts using JSON syntax, properties beginning with `mp-` are reserved as described above.

3.3 Create

To create a post, send an HTTP POST request to the Micropub endpoint indicating the type of post you are creating, as well as specifying the properties of the post. If no type is specified, the default type [h-entry] should be used. Clients and servers must support creating posts using the `x-www-form-urlencoded` syntax, and may also support creating posts using the JSON syntax.

```
h={Microformats object type}
```

e.g., `h=entry`

All parameters not beginning with "mp-" are properties of the object being created.

e.g., `content=hello+world`

To specify multiple values for a property, such as multiple categories of an h-entry, append square brackets to the property name, indicating it is an array.

e.g., `category[]=foo&category[]=bar`

Properties that accept multiple values must also accept a single value, with or without the square brackets. A complete example of a form-encoded request follows.

EXAMPLE 1

```
h=entry&content=hello+world&category[]=foo&category[]=bar
```

3.3.1 Uploading Files

To upload files, the client must check for the presence of a Media Endpoint. If there is no Media Endpoint, the client can assume that the Micropub endpoint accepts files directly, and can send the request to it directly. To upload a file to the Micropub endpoint, format the whole request as `multipart/form-data` and send the file(s) as a standard property.

For example, to upload a photo with a caption, send a request that contains three parts, named `h`, `content` and `photo`.

EXAMPLE 2

```
multipart/form-data; boundary=553d9cee2030456a81931fb708ece92c

--553d9cee2030456a81931fb708ece92c
Content-Disposition: form-data; name="h"

entry
--553d9cee2030456a81931fb708ece92c
Content-Disposition: form-data; name="content"

Hello World!
--553d9cee2030456a81931fb708ece92c
Content-Disposition: form-data; name="photo"; filename="aaronpk.png"
Content-Type: image/png
Content-Transfer-Encoding: binary

... (binary data) ...
--553d9cee2030456a81931fb708ece92c--
```

For properties that can accept a file upload (such as `photo` or `video`), the Micropub endpoint must also accept a URL value, treating that the same as if the file had been uploaded directly. The endpoint may download [Fetch] a copy of the file at the URL and store it the same way it would store the file if it had been uploaded directly. For example:

EXAMPLE 3

```
h=entry&content=hello+world&photo=https%3A%2F%2Fphotos.example.com%2F592829482876343
```

This is to support uploading files via a Media Endpoint, or by reference to other external images. See the Media Endpoint section for more information.

3.3.2 JSON Syntax

In order to support more complex values of properties, you can create a post with JSON syntax by sending the entry in the parsed Microformats 2 JSON format.

Note that in this case, you cannot also upload a file, you can only reference files by URL as described above.

When creating posts in JSON format, all values must be specified as arrays, even if there is only one value, identical to the Microformats 2 JSON format. This request is sent with a content type of `application/json`.

Note that properties beginning with `mp-` are reserved as described in Reserved Properties.

EXAMPLE 4

```
POST /micropub HTTP/1.1
Host: aaronpk.example
Content-type: application/json

{
  "type": ["h-entry"],
  "properties": {
    "content": ["hello world"],
    "category": ["foo","bar"],
```

```
    "photo": ["https://photos.example.com/592829482876343254.jpg"]
  }
}
```

Uploading a photo with alt text

To include alt text along with the image being uploaded, you can use the Microformats 2 syntax to specify both the image URL and alt text for the `photo` property. Instead of the value being simply a URL, the value is instead an object with two properties: `value` being the URL and `alt` being the text. Note that because the value of `photo` needs to be an object, we can't use form-encoded or multipart requests for this. Instead, we have to first upload the photo to the media endpoint, then reference the URL in a JSON request, as illustrated below.

EXAMPLE 5

```
POST /micropub HTTP/1.1
Host: aaronpk.example
Content-type: application/json

{
  "type": ["h-entry"],
  "properties": {
    "content": ["hello world"],
    "category": ["foo","bar"],
    "photo": [
      {
        "value": "https://photos.example.com/globe.gif",
        "alt": "Spinning globe animation"
      }
    ]
  }
}
```

WARNING

The specific method described here of specifying alt text was added with the assumption that the Microformats 2 proposal for how to represent image alt text (issue 2) will be incorporated. If the referenced issue is resolved a different way, the Micropub spec should be adjusted accordingly.

3.3.3 Nested Microformats Objects

Whenever possible, nested Microformats objects should be avoided. A better alternative is to reference objects by their URLs. The most common example is including an h-card for a venue, such as checking in to a location or tagging a photo with a person or location. In these cases, it is better to reference the object by URL, creating it first if necessary.

This technique has the advantage of ensuring that each object that is created has its own URL (each piece of data has its own link). This also gives the server an opportunity to handle each entity separately. E.g., rather than creating a duplicate of an existing venue, it may give back a link to one that was already created, possibly even merging newly received data first.

In some cases, it does not make sense for the nested object to have a URL. For example, when posting an h-measure value, there is no reason for the h-measure itself to have a URL, so this is an acceptable case to use the nested Microformats object syntax.

Nested objects require that the request be sent in JSON format, since the form-encoding syntax does not consistently support nested objects.

The example below creates a new "weight" measurement post as an h-entry with a h-measure objects to describe the weight and bodyfat values.

EXAMPLE 6

```
{
  "type": ["h-entry"],
  "properties": {
    "summary": [
      "Weighed 70.64 kg"
    ],
    "weight": [
      {
        "type": ["h-measure"],
        "properties": {
          "num": ["70.64"],
          "unit": ["kg"]
        }
      }
    ],
    "bodyfat": [
      {
        "type": ["h-measure"],
        "properties": {
          "num": ["19.83"],
          "unit": ["%"]
        }
      }
    ]
  }
}
```

3.3.4 Bidirectional Text

Natural language values in Micropub requests may contain bidirectional text. The default base direction of Micropub text values is left-to-right. The base direction of individual natural language values may be modified as described below.

When specifying bidirectional text for a natural language value, and the base direction of the text cannot be correctly identified by the first strong directional character of that text ([BIDI]), publishing clients should explicitly identify the default direction either by prefixing the value with an appropriate Unicode bidirectional control character, or by using HTML directional markup for HTML values.

Micropub servers that accept natural language values that contain bidirectional text should identify the base direction of any given natural language value by either: for plaintext values, scanning the text for the first strong directional character; or for HTML values, by utilizing directional markup, if available, or otherwise scanning for the first strong directional character not contained in a markup tag. When displaying these natural language values, the server must determine the appropriate rendering of the content according to the [BIDI] algorithm, which may necessitate

wrapping additional control characters or markup around the string prior to display, in order to apply the base direction.

3.3.5 Unrecognized Properties

If the request includes properties that the server does not recognize, it must ignore unrecognized properties and create the post with the values that are recognized.

This allows clients to post rich types of content to servers that support it, while also posting fallback content to servers that don't.

For example, a client may create a post that contains a weight measurement, with an `h-measure` value for the `weight` property, and a plaintext summary of the post in the `summary` property. Servers that don't recognize the `weight` property will simply ignore it, and will create the post as a plaintext post with the summary instead.

3.3.6 Response

When the post is created, the Micropub endpoint must return either an `HTTP 201 Created` status code or `HTTP 202 Accepted` code, and must return a `Location` header indicating the URL of the created post. [RFC2616]

EXAMPLE 7

```
HTTP/1.1 201 Created
Location: https://aaronpk.example/post/1000
```

If the endpoint chooses to process the request asynchronously rather than creating and storing the post immediately, it must return an `HTTP 202 Accepted` status code, and must also return the `Location` header. The server must do any error checking and validation of the request synchronously in order to ensure the object will be created successfully, prior to returning HTTP 202. Clients will expect the object at the indicated URL to exist at some point in the (near) future if they receive an HTTP 202 response.

If the target also provides a shortlink, or if it syndicated the post to another location, the Micropub endpoint may return additional URLs using the HTTP Link header, along with an appropriate "rel" value. For example, it can return the short URL of a post by responding with:

```
Link: <http://aaron.pk/xxxxx>; rel="shortlink"
```

or can include the location of the syndicated post with:

```
Link: <https://myfavoritesocialnetwork.example/aaronpk/xxxxxx>; rel="syndication"
```

Error Response

See the "Error Response" section below for details on how to indicate an error occurred.

3.4 Update

Micropub servers should support updating posts, including adding and removing individual properties as described in this section.

Updating entries is done with a JSON post describing the changes to make.

To update an entry, send `"action": "update"` and specify the URL of the entry that is being updated using the "url" property. The request must also include a `replace`, `add` or `delete` property (or any combination of these) containing the updates to make.

The values of each property inside the `replace`, `add` or `delete` keys must be an array, even if there is only a single value.

While it is okay to combine add/delete operations in the same request, it only makes sense to do so when they are operating on different property-value combinations. Servers may have undefined behavior if multiple operations of the same property-value combination are sent in an update request.

3.4.1 Replace

Replace all values of the property. If the property does not exist already, it is created.

EXAMPLE 8

```
{
  "action": "update",
  "url": "https://aaronpk.example/post/100",
  "replace": {
    "content": ["hello moon"]
  }
}
```

This will replace the entire entry content with the new text, leaving any other existing property of the post as is.

3.4.2 Add

If there are any existing values for this property, they are not changed, the new values are added. If the property does not exist already, it is created.

Adding a syndication URL

Use case: adding a syndication link to a post after it has been published. For example, when a client supports posting first then syndicating to MyFavoriteSocialNetwork or Wikimedia after the fact, the site needs a way to update the original post with the new syndication URL.

To add syndication URLs, include one or more URLs in the update request.

EXAMPLE 9

```
{
  "action": "update",
  "url": "https://aaronpk.example/2014/06/01/9/indieweb",
  "add": {
    "syndication": ["http://web.archive.org/web/20040104110725/https://aaronpk.examp
  }
}
```

Adding Tags

Use case: adding tags to a post after it's been created.

To add multiple values to a property (such as category), provide the new values in an array.

EXAMPLE 10

```
{
  "action": "update",
  "url": "https://aaronpk.example/2014/06/01/9/indieweb",
  "add": {
    "category": ["micropub","indieweb"]
  }
}
```

3.4.3 Remove

If the property exists, remove it. This completely removes the specified property.

EXAMPLE 11

```
{
  "action": "update",
  "url": "https://aaronpk.example/2014/06/01/9/indieweb",
  "delete": ["category"]
}
```

For properties with multiple values, such as categories, you can remove individual entries by value. If no values remain, the property is removed.

EXAMPLE 12

```
{
  "action": "update",
  "url": "https://aaronpk.example/2014/06/01/9/indieweb",
  "delete": {
    "category": ["indieweb"]
  }
}
```

3.4.4 Response

The server must respond to successful update requests with HTTP 200, 201 or 204. If the update operation caused the URL of the post to change, the server must respond with HTTP 201 and include the new URL in the HTTP `Location` header. Otherwise, the server must respond with 200 or 204, depending on whether the response body has content. No body is required in the response, but the response may contain a JSON object describing the changes that were made.

3.5 Delete

Micropub servers should support deleting posts, and may support undeleting posts.

To delete an entire entry at a URL, send a POST request containing `action=delete` and the URL of the item in the `url` property.

EXAMPLE 13

```
action=delete
&url=https://aaronpk.example/2014/06/01/9/indieweb
```

EXAMPLE 14

```
{
  "action": "delete",
  "url": "https://aaronpk.example/2014/06/01/9/indieweb"
}
```

To undelete a post, use "undelete" as the action.

EXAMPLE 15

```
action=undelete
&url=https://aaronpk.example/2014/06/01/9/indieweb
```

EXAMPLE 16

```
{
  "action": "undelete",
  "url": "https://aaronpk.example/2014/06/01/9/indieweb"
}
```

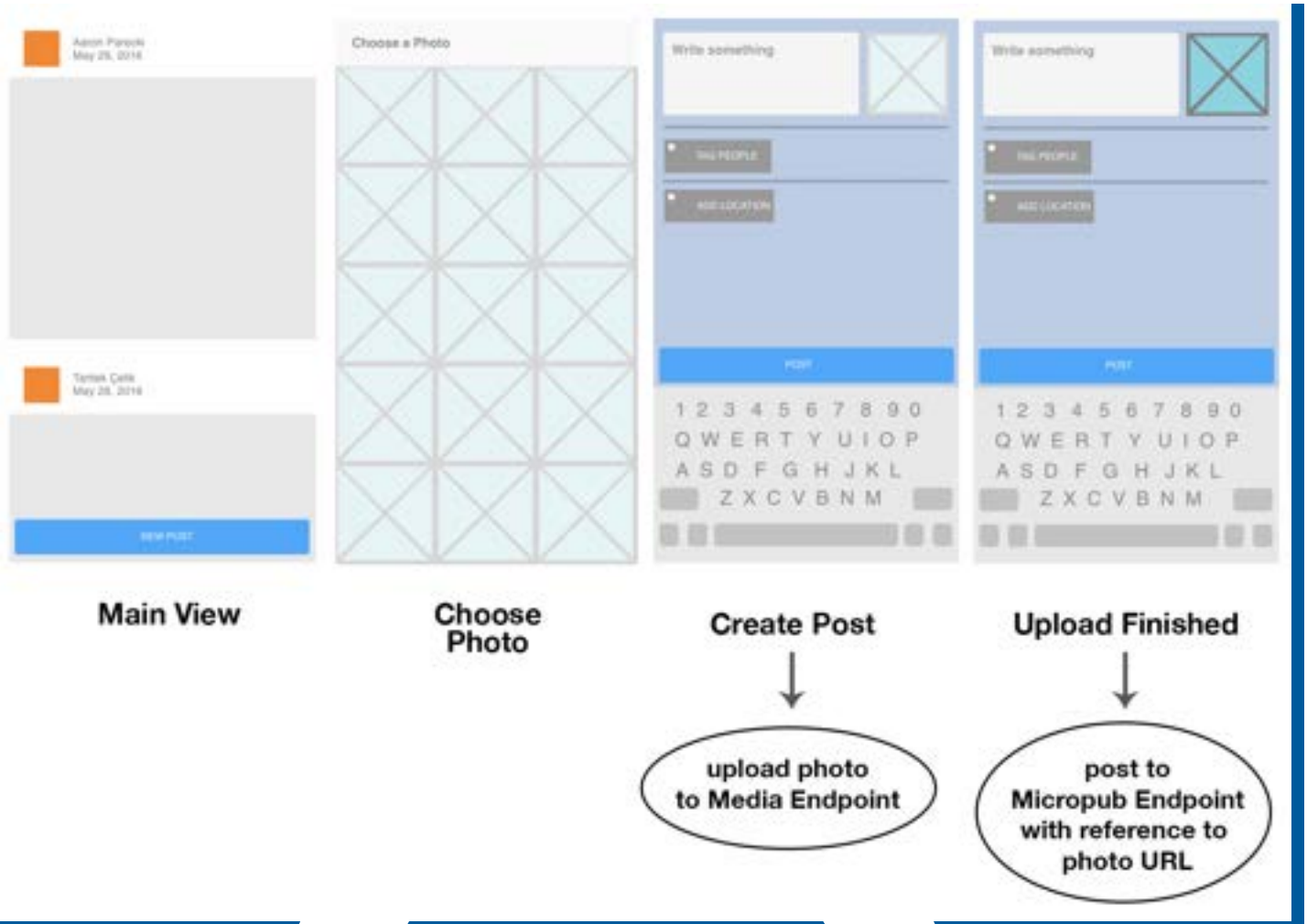
3.5.1 Response

The server must respond to successful delete and undelete requests with HTTP 200, 201 or 204. If the undelete operation caused the URL of the post to change, the server must respond with HTTP 201 and include the new URL in the HTTP `Location` header. Otherwise, the server must respond with 200 or 204, depending on whether the response body has content. No body is required in the response, but the response may contain a JSON object describing the changes that were made.

3.6 Media Endpoint

In order to provide a better user experience for Micropub applications, as well as to overcome the limitation of being unable to upload a file with the JSON syntax, a Micropub server may support a "Media Endpoint". The role of the Media Endpoint is exclusively to handle file uploads and return a URL that can be used in a subsequent Micropub request.

When a Micropub server supports a Media Endpoint, clients can start uploading a photo or other media right away while the user is still creating other parts of the post. The diagram below illustrates a user interface that demonstrates creating a photo post while the photo is uploading asynchronously.



The above user flow applies just as well to mobile apps as it does to desktop apps. In general, the user experience of an application can be improved by having more of the work done asynchronously, giving the user a chance to continue working instead of waiting for the system to finish.

3.6.1 Discovery

To advertise that the Micropub endpoint supports a Media Endpoint, the server must include a key called `media-endpoint` with a value of the full URL of the Media Endpoint in the Micropub configuration request. Clients must not assume the Media Endpoint is on the same domain as the Micropub endpoint.

EXAMPLE 17

```
GET /micropub?q=config
Authorization: Bearer xxxxxxxxx

{
  "media-endpoint": "https://media.example.com/micropub"
}
```

3.6.2 Authentication

The Media Endpoint must accept the same access tokens that the Micropub endpoint accepts.

3.6.3 Request

To upload a file to the Media Endpoint, the client sends a `multipart/form-data` request with one part named `file`. The Media Endpoint may ignore the suggested filename that the client sends.

EXAMPLE 18

```
multipart/form-data; boundary=553d9cee2030456a81931fb708ece92c

--553d9cee2030456a81931fb708ece92c
Content-Disposition: form-data; name="file"; filename="sunset.jpg"
Content-Type: image/jpeg
Content-Transfer-Encoding: binary

... (binary data) ...
--553d9cee2030456a81931fb708ece92c--
```

NOTE

To include accessibility-related information such as image alt text when creating posts that contain an image, the alt text is associated with the post itself rather than the media file. This is analogous to an HTML img tag where the img tag has a src attribute pointing to the media file and an alt attribute with the alt text. See Uploading a photo with alt text for more information.

3.6.4 Response

The Media Endpoint processes the file upload, storing it in whatever backend it wishes, and generates a URL to the file. The URL should be unguessable, such as using a UUID in the path. If the request is successful, the endpoint must return the URL to the file that was created in the HTTP `Location` header, and respond with `HTTP 201 Created`. The response body is left undefined.

EXAMPLE 19

```
HTTP/1.1 201 Created
Location: https://media.example.com/file/ff176c461dd111e6b6ba3e1d05defe78.jpg
```

The Micropub client can then use this URL as the value of e.g. the "photo" property of a Micropub request.

The Media Endpoint may periodically delete files uploaded if they are not used in a Micropub request within a specific amount of time.

3.6.4.1 Media Endpoint Error Response

The Media Endpoint should follow the same conventions for returning error responses as the Micropub endpoint, described in Error Response.

3.7 Querying

Micropub clients may need to query the Micropub endpoint to discover its capabilities, such as finding a list of syndication targets that it displays to the user, or retrieving the source of a post to display in the updating interface.

To query, make a `GET` request to the Micropub endpoint and use the `q` parameter to specify what you are querying.

NOTE

The Micropub endpoint URL may include a query string such as `?micropub=endpoint`, so in this case, Micropub clients must append the `q` parameter instead of replacing the query string.

3.7.1 Configuration

When a user initially logs in to a Micropub client, the client will want to query some initial information about the user's endpoint. The client should make a query request `q=config` to obtain initial configuration information.

The server should include the following information in the configuration response.

- The list of syndication endpoints supported, in a property called `syndicate-to` (see Syndication Targets for details on the structure of the response)
- The Media Endpoint if supported, in a property called `media-endpoint`

EXAMPLE 20

```
GET /micropub?q=config
Authorization: Bearer xxxxxxxxx
Accept: application/json

HTTP/1.1 200 OK
Content-type: application/json

{
  "media-endpoint": "https://media.example.com/micropub",
  "syndicate-to": [
    {
      "uid": "https://myfavoritesocialnetwork.example/aaronpk",
      "name": "aaronpk on myfavoritesocialnetwork",
      "service": {
        "name": "My Favorite Social Network",
        "url": "https://myfavoritesocialnetwork.example/",
        "photo": "https://myfavoritesocialnetwork.example/img/icon.png"
      },
      "user": {
        "name": "aaronpk",
        "url": "https://myfavoritesocialnetwork.example/aaronpk",
        "photo": "https://myfavoritesocialnetwork.example/aaronpk/photo.jpg"
      }
    }
  ]
}
```

Servers should support the configuration query, and returning all properties relevant to the server. If none of the properties apply to the server, the response should be an empty JSON object, `{}`.

Clients should treat unexpected responses the same as an empty response, in order to handle unexpected responses from servers that don't support this query. For example, a server that does not implement the configuration query at all may return an HTTP 400 response rather than the expected 200 response with an empty JSON object.

3.7.2 Source Content

A Micropub client can query the endpoint to return specific properties of a post. This allows a client to request only the properties it needs or knows about, supporting such uses as making an interface to add tags to a post.

Servers that support updating posts must support the source content query.

To query, make a `GET` request to the Micropub endpoint and set the `q` parameter to `source`, and include the URL of the post in the `url` parameter. The query can specify the list of properties being requested by setting one or more values for the `properties` key. If more than one is specified, use array bracket notation for each name, according to [HTML5] URL encoding.

The endpoint must return the response in [microformats2-parsing] [JSON] format, with an object named `properties`, where the keys are the names of the properties requested. If no properties are specified, then the response must include all properties, as well as a `type` property indicating the vocabulary of the post.

EXAMPLE 21

```
GET /micropub?q=source&properties[]=published&properties[]=category&url=https://aaro
Authorization: Bearer xxxxxxxxx
Accept: application/json

HTTP/1.1 200 OK
Content-type: application/json

{
  "properties": {
    "published": ["2016-02-21T12:50:53-08:00"],
    "category": [
      "foo",
      "bar"
    ]
  }
}
```

EXAMPLE 22

```
GET /micropub?q=source&url=https://aaronpk.example/post/1000
Authorization: Bearer xxxxxxxxx
Accept: application/json

HTTP/1.1 200 OK
Content-type: application/json

{
  "type": ["h-entry"],
  "properties": {
    "published": ["2016-02-21T12:50:53-08:00"],
    "content": ["Hello World"],
    "category": [
      "foo",
      "bar"
    ]
  }
}
```

HTML Content

If the source of the post was written as HTML content, then the endpoint must return the `content` property as an object containing an `html` property. Otherwise, the endpoint must return a string value for the `content` property, and the client will treat the value as plain text. This matches the behavior of the values of properties in [microformats2-parsing].

Below is an example of requesting the content of a post which was authored as HTML.

EXAMPLE 23

```
GET /micropub?q=source&properties=content&url=https://aaronpk.example/post/1000
Authorization: Bearer xxxxxxxxx
Accept: application/json

HTTP/1.1 200 OK
Content-type: application/json

{
  "properties": {
    "content": [
      {
        "html": "<b>Hello</b> <i>World</i>"
      }
    ]
  }
}
```

3.7.3 Syndication Targets

To return a list of supported syndication targets, set the `q` parameter to `syndicate-to`.

```
GET /micropub?q=syndicate-to
```

The endpoint must return the response in [JSON] format, with a key of `syndicate-to` and the values being objects descriptive of the supported syndication targets. If no endpoints are defined, the value of the `syndicate-to` object should be an empty JSON array `[]`.

At a minimum, a syndication target must have a `uid` and human-readable `name` property. The `uid` can be any identifier unique to the Micropub endpoint and is not intended to be visible to the end user. The `name` is intended to be visible to the user in the posting interface, and should be descriptive of the syndication destination.

A syndication target may optionally include additional details about the service or user account at the syndication destination, indicated with a property called `service` and `user` respectively. Both service and user objects must contain a `name` property, intended to be displayed to the user, and may contain a `url` and `photo` property. The client can use the name and photo to improve the display of the syndication options to the user.

EXAMPLE 24

```
GET /micropub?q=syndicate-to
Authorization: Bearer xxxxxxxxx
Accept: application/json

HTTP/1.1 200 OK
Content-type: application/json

{
  "syndicate-to": [
```

```
    {
      "uid": "https://archive.org/",
      "name": "archive.org"
    },
    {
      "uid": "https://wikimedia.org/",
      "name": "WikiMedia"
    },
    {
      "uid": "https://myfavoritesocialnetwork.example/aaronpk",
      "name": "aaronpk on myfavoritesocialnetwork",
      "service": {
        "name": "My Favorite Social Network",
        "url": "https://myfavoritesocialnetwork.example/",
        "photo": "https://myfavoritesocialnetwork.example/img/icon.png"
      },
      "user": {
        "name": "aaronpk",
        "url": "https://myfavoritesocialnetwork.example/aaronpk",
        "photo": "https://myfavoritesocialnetwork.example/aaronpk/photo.jpg"
      }
    }
  ]
}
```

At a minimum, a syndication destination must have `uid` and `name` properties. The `uid` property is opaque to the client, and is the value the client sends in the Micropub request to indicate the targets to syndicate to. The `name` property is the value the client can display to the user. The name should be unambiguously descriptive of the destination, so that if for example there are two destinations on the same service, the name can disambiguate them.

The Micropub server may also include additional information about the destination service and user account. This is accomplished with the additional properties `service` and `user`, both of which can have three properties, `name` (a human-readable name), `url` and `photo` (both URLs).

The client may use the service and user properties to enhance the display of the syndication options, for example by including the user or service photos on the syndication buttons.

Servers that support syndicating posts should support the `syndicate-to` query.

3.7.4 Query Extensions

Clients and servers wishing to extend the functionality of the query action are encouraged to brainstorm and document implementations at indieweb.org/Micropub-extensions.

3.8 Error Response

If there was an error with the request, the endpoint must return an appropriate HTTP status code, typically 400, 401, or 403, and may include a description of the error. If an error body is returned, the response body must be encoded as a [JSON] object and include at least a single property named `error`. The following error codes are defined:

- HTTP 403: `"error":"forbidden"` - The authenticated user does not have permission to perform this request.
- HTTP 401: `"error":"unauthorized"` - No access token was provided in the request. Note that this is different from the HTTP 403 response, as the 403 response should only be used when an access token is provided and the user does not have permission to perform the request.

- HTTP 401: `"error":"insufficient_scope"` - The scope of this token does not meet the requirements for this request. The client may wish to re-authorize the user to obtain the necessary scope. The response may include the `"scope"` attribute with the scope necessary to successfully perform this request.
- HTTP 400: `"error":"invalid_request"` - The request is missing a required parameter, or there was a problem with a value of one of the parameters

Clients should treat unexpected error strings as a generic error. The response body may also contain an `error_description` property with a human-readable description of the error message, used to assist the client developer in understanding the error. This is not meant to be shown to the end user.

See the OAuth 2 Bearer Token [RFC6750] spec for more details on the how to return error responses.

EXAMPLE 25

```
GET /micropub?q=source&url=https://aaronpk.example/post/404
Authorization: Bearer xxxxxxxx

HTTP/1.1 400 Bad Request
Content-type: application/json

{
  "error": "invalid_request",
  "error_description": "The post with the requested URL was not found"
}
```

4. Vocabulary

The vocabularies used in Micropub requests should be the vocabularies defined by [Microformats2]. If the Microformats2 vocabulary is used, clients and servers must support at least the [h-entry] vocabulary. Other vocabularies with widespread usage include [h-event] and [h-card]. When creating objects, the vocabulary of the object is indicated in the parameter `h`, or `type` in the JSON syntax. If no type is provided, the server should assume a default value of `entry`.

4.1 Examples of Creating Objects

This section is non-normative.

To indicate the object being created, use a property called `h`, (which would never be the name of a property of a Microformats object), followed by the name of the Microformats object. Examples:

- `h=entry`
- `h=card`
- `h=event`
- `h=cite`

The following properties may be included in a request to create a new [h-entry]:

- name
- summary
- content
- published
- updated
- category

- location
  - A plaintext string describing the location
  - As a Geo URI [RFC5870], for example: `geo:45.51533714,-122.646538633`
  - As a URL that contains an [h-card]
  - As a nested [h-adr] object
- in-reply-to
- like-of
- repost-of
- syndication - Pass one or more URLs pointing to places where this entry already exists. Can be used for importing existing content to a site.
- mp-syndicate-to = https://myfavoritesocialnetwork.example/aaronpk, https://archive.org/, etc.
  - This property is giving a command to the Micropub endpoint, rather than just creating data, so it uses the mp- prefix.

4.1.1 New Note

Posting a new note with tags, syndicating to myfavoritesocialnetwork:

- content
- category
- published (optional, defaults to "now" if not present. Useful for writing offline and syncing later.)
- mp-syndicate-to

EXAMPLE 26

```
POST /micropub HTTP/1.1
Host: aaronpk.example
Authorization: Bearer XXXXXXXXXXX
Content-type: application/x-www-form-urlencoded; charset=utf-8

h=entry
&content=My+favorite+of+the+%23quantifiedself+trackers%2C+finally+released+their+off:
&category[]=quantifiedself&category[]=api
&mp-syndicate-to=https://myfavoritesocialnetwork.example/aaronpk
```

Minimal Example

EXAMPLE 27

```
POST /micropub HTTP/1.1
Host: aaronpk.example
Content-type: application/x-www-form-urlencoded; charset=utf-8
Authorization: Bearer XXXXXXX

h=entry
&content=Hello+World
```

EXAMPLE 28

```
curl https://aaronpk.example/micropub -d h=entry -d "content=Hello World" -H "Author.
```

4.1.2 New Reply

Posting a new note with tags, syndicating to myfavoritesocialnetwork:

- content
- in-reply-to
- published
- mp-syndicate-to

EXAMPLE 29

```
POST /micropub HTTP/1.1
Host: aaronpk.example
Authorization: Bearer XXXXXXXXXXX
Content-type: application/x-www-form-urlencoded; charset=utf-8

h=entry
&content=%40BarnabyWalters+My+favorite+for+that+use+case+is+Redis.
&in-reply-to=https://waterpigs.example/notes/4S0LMw/
&mp-syndicate-to=https://myfavoritesocialnetwork.example/aaronpk
```

4.1.3 New Article with HTML

Posting a new article with HTML content. Note that in this case, the `content` property is sent as an object containing the key `html`. This corresponds with the Microformats 2 syntax for indicating the parsed value contains HTML. Because `content` is an object, this request must be sent in JSON format.

EXAMPLE 30

```
POST /micropub HTTP/1.1
Host: aaronpk.example
Content-type: application/json

{
  "type": ["h-entry"],
  "properties": {
    "name": ["Itching: h-event to iCal converter"],
    "content": [
      {"html": "Now that I've been <a href=\"https://aaronparecki.com/events\">creati
    ],
    "category": [
      "indieweb", "p3k"
    ]
  }
}
```

4.1.4 New Article with Embedded Images

To create an article containing embedded images, the HTML of the article should include `<img>` tags with the URLs of the images returned after uploading them to the Media Endpoint.

Upload images to the Media Endpoint

First, upload one or more images to the Media Endpoint. Typically this will happen before the article is published, such as when the user is authoring the article in a visual editor. The editor can upload the images to the Media Endpoint as soon as the user drags them into the interface, and then embed the images into the editor via the resulting URL. See Uploading to the Media Endpoint for an example of this request.

The response after uploading the file to the Media Endpoint will be a URL that the client can use when posting the article.

EXAMPLE 31

```
HTTP/1.1 201 Created
Location: https://media.example.com/file/ff176c461dd111e6b6ba3e1d05defe78.jpg
```

The client can then use this URL to embed this image in an article.

EXAMPLE 32

```
POST /micropub HTTP/1.1
Host: aaronpk.example
Content-type: application/json

{
  "type": ["h-entry"],
  "properties": {
    "content": [
      {"html":"<p>Hello World</p><p><img src=\"https://media.example.com/file/ff176c4
    ]
  }
}
```

4.1.5 Posting Files

When a Micropub request includes a file, the entire request is sent in `multipart/form-data` encoding, and the file is named according to the property it corresponds with in the vocabulary, either `audio`, `video` or `photo`. A request may include one or more of these files.

For example, a service may post a video in the `video` property as well as a single frame preview of the video in the `photo` property.

In PHP, these files are accessible using the $_FILES array:

```
$_FILES['video']
$_FILES['photo']
$_FILES['audio']
```

Note that there is no way to upload a file when the request body is JSON encoded. Instead, you can first send the photo to the Media Endpoint and then use the resulting URL in the JSON post.

The Micropub endpoint may store the file directly, or make an external request to upload it to a different backend storage, such as Amazon S3.

5. Authentication and Authorization

5.1 Authentication

Micropub requests must be authenticated by including a Bearer Token in either the HTTP header or a form-encoded body parameter as described in the OAuth Bearer Token RFC [RFC6750]. Micropub servers must accept the token using both methods.

HTTP Header

```
Authorization: Bearer XXXXXXXXX
```

Form-Encoded Body Parameter

```
access_token=XXXXXXXXX
```

5.2 Authorization

An app that wants to post to a user's Micropub endpoint will need to obtain authorization from the user in order to get an access token. Authorization should be handled via the OAuth 2.0 [RFC6749] protocol. Applications may use the [IndieAuth] extension which supports endpoint discovery from a URL. See Obtaining an Access Token for more details.

5.3 Endpoint Discovery

Micropub defines a link relation value of `micropub` to link to a website's Micropub endpoint from the profile page identifying the user.

A Micropub client should be able to be configured by discovering the Micropub endpoint for the user via this link relation.

The client looks for a `<link rel="micropub">` tag in the HTML head of the URL used to authenticate, or an `HTTP Link` header with a rel value of `micropub`.

HTTP Link Header

```
Link: <https://aaronpk.example/micropub>; rel="micropub"
```

HTML Link Tag

```
<link rel="micropub" href="https://aaronpk.example/micropub">
```

5.4 Scope

The Micropub server must require the bearer token to include at least one scope value, in order to ensure posts cannot be created by arbitrary tokens.

The client may request one or more scopes during the authorization request. It does this according to standard OAuth 2.0 [RFC6749] techniques, by passing a space-separated list of scope names in the authorization request. See RFC6749 Section 3.3.

The authorization server must indicate to the user any scopes that are part of the request, whether or not the authorization server recognizes the scopes. The authorization server may also allow the user to add or remove scopes

that the client requests.

Most Micropub servers require clients to obtain the "create" scope in order to create posts. However, some servers may require more granular scope, such as "delete", "update", or "create:video", in order to limit the abilities of various clients. See Scope for more details and a list of all currently used values for scope.

6. Security Considerations

6.1 External Content

Micropub endpoints that accept photos or other media may encounter a request from a client that contains a URL to a file on a domain other than the Micropub endpoint or Media Endpoint, such as if the client provides its own mechanism for uploading files. One defensive way Micropub servers can implement accepting URL values is to only download and store media from a list of trusted domains, such as the Micropub server's domain and its Media Endpoint, and store just the URL to the file otherwise.

6.2 Security and Privacy Review

These questions provide an overview of security and privacy considerations for this specification as guided by Self-Review Questionnaire: Security and Privacy ([security-privacy-questionnaire]).

Does this specification deal with personally-identifiable information?

Micropub enables users to create and edit posts, so users may enter personally identifiable information when creating content. Micropub clients are aware of the URL of the Micropub endpoint at which they are managing content, and this URL may be personally identifiable or may be a shared domain.

Does this specification deal with high-value data?

This specification uses OAuth 2.0 Bearer Tokens, which are considered "high-value" according to the Security and Privacy questionnaire. You can refer to the OAuth 2.0 Bearer Token (RFC6750) Security Considerations section for more information.

Does this specification introduce new state for an origin that persists across browsing sessions?

No, Micropub does not provide any mechanism to persist data across browsing sessions.

Does this specification expose persistent, cross-origin state to the web?

This spec allows clients to create posts, and the server may make the new URL publicly viewable.

Does this specification expose any other data to an origin that it doesn't currently have access to?

No

Does this specification enable new script execution/loading mechanisms?

No

Does this specification allow an origin access to a user's location?

No

Does this specification allow an origin access to sensors on a user's device?

No

Does this specification allow an origin access to aspects of a user's local computing environment?

No

Does this specification allow an origin access to other devices?

No

Does this specification allow an origin some measure of control over a user agent's native UI?

No

Does this specification expose temporary identifiers to the web?

No

Does this specification distinguish between behavior in first-party and third-party contexts?

No

How should this specification work in the context of a user agent's "incognito" mode?

If the user agent is a Micropub client, the client should "forget" any access tokens and Micropub endpoints associated with the user's browsing session when in "incognito" mode.

Does this specification persist data to a user's local device?

Micropub clients will need to store the user's Micropub endpoint and access token. When the user clears their browsing data, the client should remove any stored credentials, as well as any cached URLs from the Media Endpoint.

Does this specification allow downgrading default security characteristics?

No

7. IANA Considerations

The link relation type below has been registered by IANA per Section 6.2.1 of [RFC5988]:

Relation Name:

micropub

Description:

Used for discovery of a Micropub endpoint which can be used to create posts according to the Micropub protocol.

Reference:

W3C Micropub Specification (https://www.w3.org/TR/micropub/)

A. Resources

This section is non-normative.

- FAQ
- Creating a Micropub Endpoint
- Obtaining an Access Token
- Brainstorming

A.1 Implementations

This section is non-normative.

You can find a list of Micropub implementations on micropub.net

B. Acknowledgements

The editor wishes to thank the IndieWeb community and other implementers for their support, encouragement and enthusiasm. In particular, the editor wishes to thank Amy Guy, Barry Frost, Benjamin Roberts, Chris Webber, Christian Weiske, David Shanske, Emma Kuo, Kyle Mahan, Jeena Paradies, Malcolm Blaney, Martijn van der Ven, Marty McGuire, Mike Taylor, Pelle Wessman, Ryan Barrett, Sandro Hawke, Sven Knebel, and Tantek Çelik.

C. Change Log

This section is non-normative.

C.1 Changes from 13 April 2017 PR to this version

This section lists changes from the 13 April 2017 PR to this version.

Non-Normative Changes

- Added text to clarify that update requests to replace a property do not modify other properties of the post.
- Removed `q` from the list of reserved POST body parameters since it is not used in POST requests.
- Clarified that `mp-` properties are also reserved in JSON syntax.
- Clarified that servers may have undefined behavior if multiple operations of the same property-value combination are sent in an update request
- Fixed typo in discovery section
- Added link to the specific section of OAuth 2.0 RFC6749 about the scope value

C.2 Changes from 18 October 2016 CR to 13 April 2017 PR

This section lists changes from the 18 October 2016 CR to the 13 April 2017 PR.

Non-Normative Changes

- Corrected conformance class description of servers handling `multipart/form-data` requests for accepting media
- Add background referring to MetaWeblog and AtomPub prior art
- Clarification of condition under which a client can try uploading a file
- Editorial changes to introduction and abstract
- Expand acknowledgements
- Use consistent spelling and `<code>` tag for `x-www-form-urlencoded` and `multipart/form-data`
- Added note under security considerations about handling files referenced by URL

C.3 Changes from 16 August 2016 CR to 18 October 2016 CR

This section lists changes from the 16 August 2016 CR to the 18 October 2016 CR.

Normative Changes

- New method for specifying image alt text on properties that support images
- Add normative text describing the configuration and syndicate-to response
- Drop `not_found` from the list of error codes, changed from requiring a specific error code string to recommending these three and allowing the server to return others
- Remove "note" designation from sentence about storing access token

Non-Normative Changes

- Rephrase paragraph on HTTP 202 response
- Add explicit note that JSON posts require content-type header
- Add example of including embedded images when posting an article
- Add explicit utf-8 charset to form-encoded examples
- Add note about how to specify base direction of bidirectional text
- Document conformance requirements of querying
- Explicitly documented config response when empty
- Note that clients should treat unknown config responses as an empty response
- Editorial clarifications around update operations, mp-* commands, and bidi text
- Add explicit note of using numeric offsets in form-encoded arrays
- Add reference to error response section under Media Endpoint
- Updated intro paragraph to link to Social Web Protocols
- Add Security and Privacy Questionnaire

- Clarify form-encoded usage and limitations
- Added direction on extending Micropub with new `mp-` and `q=` functionality

C.4 Changes from 13 July 2016 WD to 16 August 2016 CR

This section lists changes from the 13 July 2016 WD to the 16 August 2016 CR.

- Clarified text around IndieAuth references to ensure they are non-normative
- Add reference to Fetch for downloading contents of a URL

C.5 Changes from 21 June 2016 WD to 13 July 2016 WD

This section lists changes from the 21 June 2016 WD to the 13 July 2016 WD.

- Clarified requirement of supporting file uploads directly or supporting a Media Endpoint
- Updated indiewebcamp.com references to indieweb.org due to redirects
- Rename `mp-action` to `action` since updates are only JSON requests
- Added link to test suite in document header
- Added section describing how to submit implementation reports
- Added conformance criteria and classes section
- Updated example URLs
- Added list of features for exit criteria
- Added note about the accessibility group review

C.6 Changes from 04 May 2016 WD to 21 June 2016 WD

This section lists changes from the 04 May 2016 WD to the 21 June 2016 WD.

- Clarified use of arrays vs single values in requests
- Drop form-encoded syntax for update requests
- Removed warning about lack of interoperable implementations based on new implementations
- Updated list of implementations
- Added explicit note about not storing the `access_token` value
- Added details and diagram on using a Media Endpoint for handling file uploads separately
- Expanded introduction and background
- Moved list of implementations out of this document onto https://micropub.net/implementations

C.7 Changes from 01 March 2016 WD to 04 May 2016 WD

This section lists changes from the 01 March 2016 WD to the 04 May 2016 WD.

- Removed extra nesting of `properties` key in updates
- Non-normative typo fixes
- Added HTTP 201 with Location header as a response for updates
- Updated `syndicate-to` response to include more details of the destination

C.8 Changes from 28 January 2016 FPWD to 01 March 2016 WD

This section lists changes from the 28 January 2016 FPWD to the 01 March 2016 WD.

- Corrected references to be normative where appropriate
- Added new Conformance section summarizing conformance requirements for Micropub Clients and Servers

- Clarified that a server should create the post with any properties recognized when it receives a request with unrecognized properties
- Added a section on querying the Micropub endpoint for the source of specific properties, to better support editing
- Added new "Vocabulary" section which normatively references Microformats 2 vocabularies
- Micropub servers may include a JSON object describing the changes that were applied for Updates
- Added a section listing specific error responses, also referencing the OAuth 2 Bearer Token spec for other error responses
- If there is a response body, it should be JSON
- Default to create an h-entry if no type is specified
- Clarified posting files and sending HTML content
- Reorganized "How To" section into an "Authentication and Authorization" section
- Fixed some examples of HTTP requests that were missing the response headers
- Updated references from Microformats to Microformats 2

D. References

D.1 Normative references

[BIDI]
Unicode Bidirectional Algorithm. Mark Davis; Aharon Lanin; Andrew Glass. Unicode Consortium. 18 May 2016. Unicode Standard Annex #9. URL: http://www.unicode.org/reports/tr9/tr9-35.html

[Fetch]
Fetch Standard. Anne van Kesteren. WHATWG. Living Standard. URL: https://fetch.spec.whatwg.org/

[HTML5]
HTML5. Ian Hickson; Robin Berjon; Steve Faulkner; Travis Leithead; Erika Doyle Navara; Theresa O'Connor; Silvia Pfeiffer. W3C. 28 October 2014. W3C Recommendation. URL: https://www.w3.org/TR/html5/

[JSON]
The application/json Media Type for JavaScript Object Notation (JSON). D. Crockford. IETF. July 2006. Informational. URL: https://tools.ietf.org/html/rfc4627

[RFC2119]
Key words for use in RFCs to Indicate Requirement Levels. S. Bradner. IETF. March 1997. Best Current Practice. URL: https://tools.ietf.org/html/rfc2119

[RFC2616]
Hypertext Transfer Protocol -- HTTP/1.1. R. Fielding; J. Gettys; J. Mogul; H. Frystyk; L. Masinter; P. Leach; T. Berners-Lee. IETF. June 1999. Draft Standard. URL: https://tools.ietf.org/html/rfc2616

[RFC5988]
Web Linking. M. Nottingham. IETF. October 2010. Proposed Standard. URL: https://tools.ietf.org/html/rfc5988

[RFC6749]
The OAuth 2.0 Authorization Framework. D. Hardt, Ed.. IETF. October 2012. Proposed Standard. URL: https://tools.ietf.org/html/rfc6749

[RFC6750]
The OAuth 2.0 Authorization Framework: Bearer Token Usage. M. Jones; D. Hardt. IETF. October 2012. Proposed Standard. URL: https://tools.ietf.org/html/rfc6750

[h-entry]
h-entry. Tantek Çelik. microformats.org. Living Specification. URL: http://microformats.org/wiki/h-entry

[microformats2-parsing]
Microformats2 Parsing. Tantek Çelik. microformats.org. Living Specification. URL: http://microformats.org/wiki/microformats2-parsing

D.2 Informative references

[IndieAuth]
IndieAuth. Aaron Parecki. indieweb.org. Living Specification. URL: https://indieweb.org/IndieAuth-spec

[Microformats2]
Microformats 2. Tantek Çelik. microformats.org. Living Specification. URL: http://microformats.org/wiki/microformats2

[RFC5870]
A Uniform Resource Identifier for Geographic Locations ('geo' URI). A. Mayrhofer; C. Spanring. IETF. June 2010. Proposed Standard. URL: https://tools.ietf.org/html/rfc5870

[h-adr]
h-adr. Tantek Çelik. microformats.org. Living Specification. URL: http://microformats.org/wiki/h-adr

[h-card]
h-card. Tantek Çelik. microformats.org. Living Specification. URL: http://microformats.org/wiki/h-card

[h-event]
h-event. Tantek Çelik. microformats.org. Living Specification. URL: http://microformats.org/wiki/h-event

[security-privacy-questionnaire]
Self-Review Questionnaire: Security and Privacy. Mike West. W3C. 10 December 2015. W3C Note. URL: https://www.w3.org/TR/security-privacy-questionnaire/

[social-web-protocols]
Social Web Protocols. Amy Guy. W3C. 4 May 2017. W3C Working Draft. URL: https://www.w3.org/TR/social-web-protocols/

↑